Data Structures Lab Exercises Solutions

Lab Exercise 3 (Recursion & Arrays) & Lab Exercise 4 (Queues)

Name : Yuvaraj R
Reg No: 26MCS1019

Question 1: Lab Exercise 3 - Question 1: Largest Rectangular Area in Histogram

Problem Description: Find the area of the largest rectangle in a histogram using simple nested iteration.

C Solution Code:

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

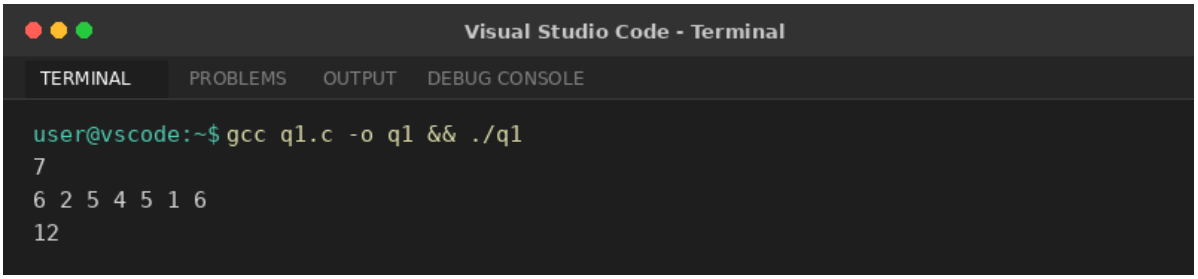
    int h[100];
    for (int i = 0; i < n; i++) {
        scanf("%d", &h[i]);
    }

    int max_area = 0;

    for (int i = 0; i < n; i++) {
        int min_height = h[i];
        for (int j = i; j < n; j++) {
            if (h[j] < min_height) {
                min_height = h[j];
            }
            int width = j - i + 1;
            int area = min_height * width;
            if (area > max_area) {
                max_area = area;
            }
        }
    }

    printf("%d\n", max_area);
    return 0;
}
```

VS Code Terminal Execution Output:



Question 2: Lab Exercise 3 - Question 2: Tower of Hanoi Non-Recursive

Problem Description: Solve the Tower of Hanoi puzzle using a simple bitwise iterative algorithm.

C Solution Code:

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int total_moves = (1 << n) - 1;
    char source = 'A', aux = 'B', dest = 'C';

    if (n % 2 == 0) {
        char temp = dest;
        dest = aux;
        aux = temp;
    }

    for (int i = 1; i <= total_moves; i++) {
        int disk = 0;
        int temp = i;
        while ((temp & 1) == 0) {
            disk++;
            temp >>= 1;
        }
        disk += 1;

        int from_idx = (i & (i - 1)) % 3;
        int to_idx = ((i | (i - 1)) + 1) % 3;

        char pegs[3];
        pegs[0] = source;
        pegs[1] = dest;
        pegs[2] = aux;

        printf("Move disk %d from %c to %c\n", disk, pegs[from_idx], pegs[to_idx]);
```

```
    }

    printf("Total number of moves: %d\n", total_moves);
    return 0;
}
```

VS Code Terminal Execution Output:



Question 3: Lab Exercise 4 - Question 1: Customer Queue at Supermarket Checkout

Problem Description: Simulate a supermarket checkout queue supporting enqueue, dequeue, display, and exit.

C Solution Code:

```
#include <stdio.h>


int queue[100];
int front = 0, rear = 0;

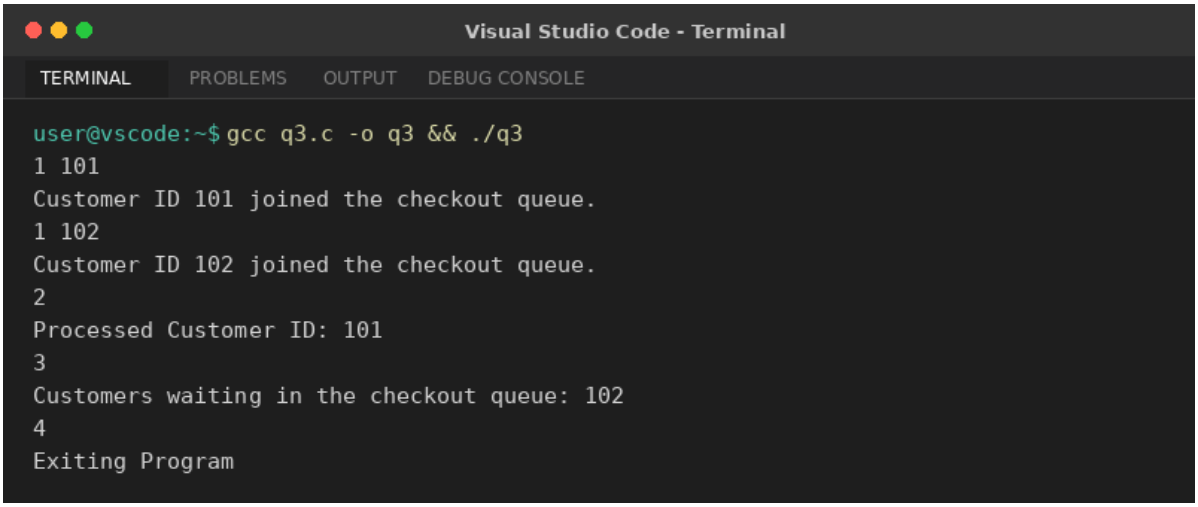
void enqueue(int id) {
    queue[rear++] = id;
    printf("Customer ID %d joined the checkout queue.\n", id);
}

void dequeue() {
    if (front == rear) {
        printf("Queue is empty\n");
    } else {
        printf("Processed Customer ID: %d\n", queue[front++]);
    }
}
```

```c
void display() {
    if (front == rear) {
        printf("Queue is empty\n");
    } else {
        printf("Customers waiting in the checkout queue: ");
        for (int i = front; i < rear; i++) {
            printf("%d ", queue[i]);
        }
        printf("\n");
    }
}

int main() {
    int choice, id;
    while (1) {
        if (scanf("%d", &choice) != 1) break;
        if (choice == 1) {
            scanf("%d", &id);
            enqueue(id);
        } else if (choice == 2) {
            dequeue();
        } else if (choice == 3) {
            display();
        } else if (choice == 4) {
            printf("Exiting Program\n");
            break;
        }
    }
    return 0;
}
```

VS Code Terminal Execution Output:



Question 4: Lab Exercise 4 - Question 2: Supermarket Checkout Queue (Fixed Size Array)

Problem Description: Implement a supermarket checkout queue using a fixed size array of 10 capacity.

C Solution Code:

```c
#include <stdio.h>

#define MAX 10

int main() {
    int queue[MAX];
    int front = 0, rear = 0;
    int n;

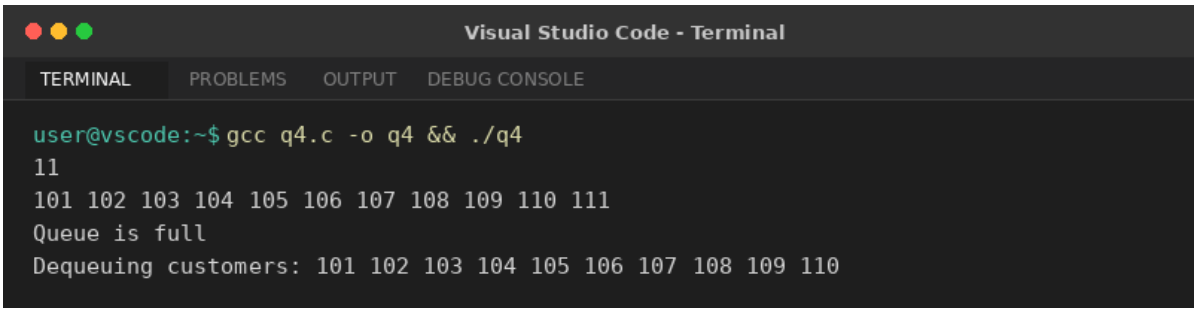
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        int id;
        scanf("%d", &id);
        if (rear < MAX) {
            queue[rear++] = id;
        } else {
            printf("Queue is full\n");
        }
    }

    printf("Dequeuing customers: ");
    while (front < rear) {
        printf("%d ", queue[front++]);
    }
    printf("\n");

    return 0;
}
```

VS Code Terminal Execution Output:



Question 5: Lab Exercise 4 - Question 3: Array Implementation of Queue

Problem Description: Demonstrate basic queue enqueue and single dequeue using an array.

C Solution Code:

```c
#include <stdio.h>

int main() {
    int queue[100];
    int front = 0, rear = 0;
    int n;

    if (scanf("%d", &n) != 1) return 0;

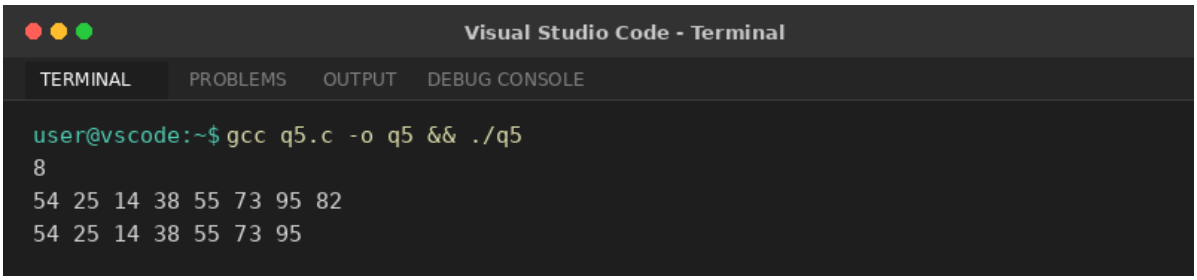
    for (int i = 0; i < n; i++) {
        scanf("%d", &queue[rear++]);
    }

    // Dequeue one element
    if (front < rear) {
        front++;
    }

    // Print remaining
    for (int i = front; i < rear; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");

    return 0;
}
```

VS Code Terminal Execution Output:



Question 6: Lab Exercise 4 - Question 4: Linked List Implementation of Queue

Problem Description: Implement a FIFO queue using a singly linked list with Enqueue, Dequeue, and Display functions.

C Solution Code:

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* front = NULL;
struct Node* rear = NULL;

void enqueue(int val) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = val;
    temp->next = NULL;
    if (rear == NULL) {
        front = rear = temp;
        return;
    }
    rear->next = temp;
    rear = temp;
}

void display() {
    struct Node* temp = front;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

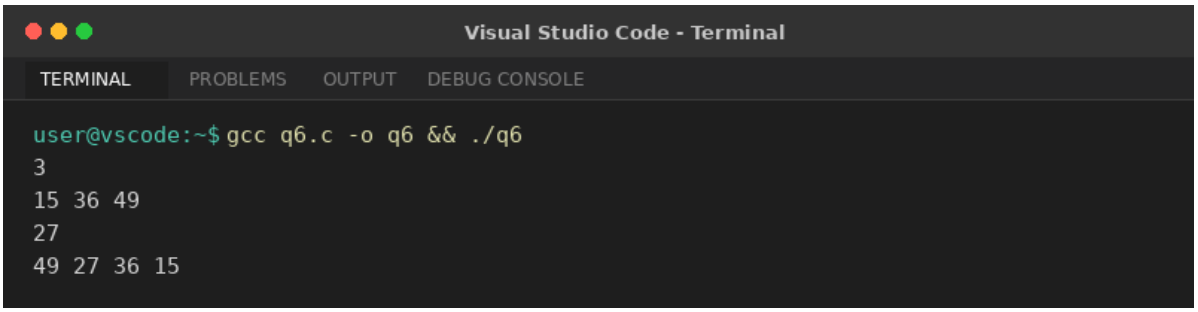
int main() {
    int n, val, extra;
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        enqueue(val);
    }

    scanf("%d", &extra);
    enqueue(extra);

    display();
    return 0;
}
```

VS Code Terminal Execution Output:



Question 7: Lab Exercise 4 - Question 5: Queue Enqueue and Display

Problem Description: Enqueue a list of integers into a queue and display them in FIFO order.

C Solution Code:

```c
#include <stdio.h>

int main() {
    int queue[100];
    int front = 0, rear = 0;
    int n;

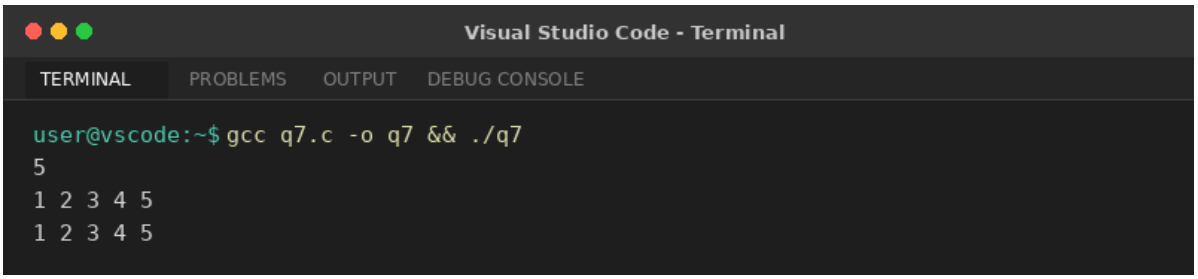
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &queue[rear++]);
    }

    for (int i = front; i < rear; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");

    return 0;
}
```

VS Code Terminal Execution Output:



Question 8: Lab Exercise 4 - Question 6: Linked List Node Insertion & Queue Output

Problem Description: Dynamically insert elements at the front of a linked list and display queue state after each step.

C Solution Code:

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

void insertFront(int val) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = val;
    temp->next = head;
    head = temp;
    printf("Node Inserted\n");
}

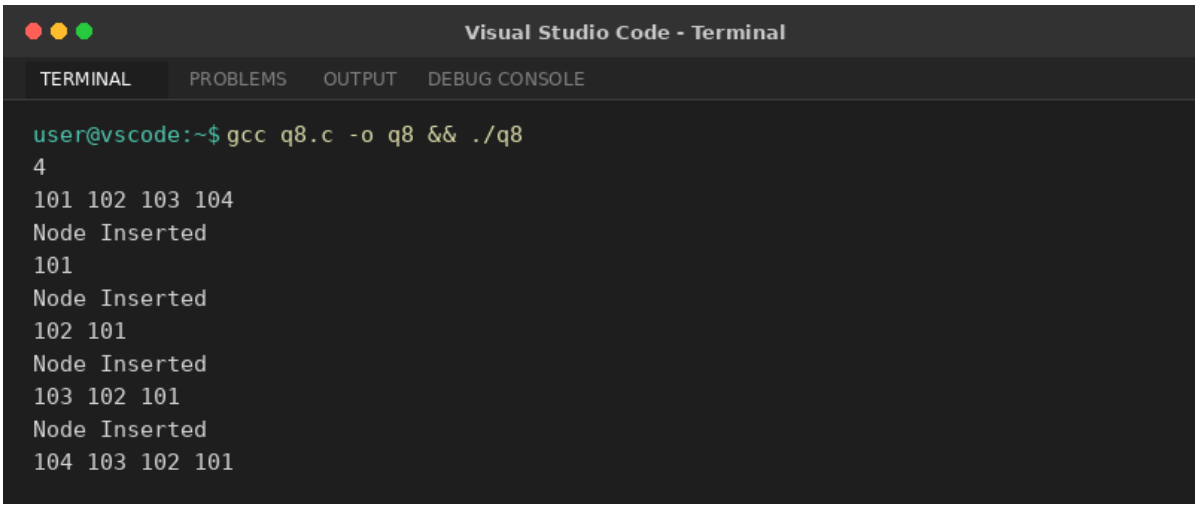
void display() {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n, val;
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        insertFront(val);
        display();
    }

    return 0;
}
```

VS Code Terminal Execution Output:


Visual Studio Code - Terminal
TERMINAL
PROBLEMS
OUTPUT
DEBUG CONSOLE
user@vscode:~$ gcc q8.c -o q8 && ./q8
4
101 102 103 104
Node Inserted
101
Node Inserted
102 101
Node Inserted
103 102 101
Node Inserted
104 103 102 101